

Project Report

RentCluster – Distributed Rental Management System

Course:

Big Data and Elements of Distribution

Realised by:

Ahmed Boudokhane (ER-2203)

Ahmet Yasin Karaman (ER-2190)

Instructor:

mgr inż. Adam Godziński

Poznań University of Technology
Faculty of Computing and Telecommunications

Academic Year 2025–2026



POLITECHNIKA POZNAŃSKA

1 Project Description

RentCluster is a distributed rental management system developed as part of the *Big Data and Elements of Distribution* course. The objective of the project was to design and implement a reservation platform capable of handling concurrent booking requests while maintaining data consistency in a distributed environment.

The application was developed using Java and Apache Cassandra. A graphical user interface was implemented using Java Swing, while Cassandra was deployed as a three-node cluster using Docker containers. The project focuses on solving one of the most common problems in booking systems: preventing multiple users from reserving the same property for the same date.

Unlike traditional relational database systems, Apache Cassandra is designed for scalability, high availability, and write-intensive workloads. To take advantage of these characteristics, reservations are stored on a day-by-day basis rather than as date ranges. This denormalized approach simplifies availability checks and improves write performance.

2 Database Schema

The database is organized around two main entities: properties and reservations.

2.1 Properties

The properties table stores information about rental units available for booking.

```
properties
-----
```

```
property_id UUID PRIMARY KEY
property_name TEXT
location TEXT
```

2.2 Reservations

The reservations table stores booking information at daily granularity.

```
reservations
-----
```

```
property_id UUID
reservation_date DATE
customer_name TEXT
```

```
PRIMARY KEY ((property_id), reservation_date)
```

The partition key is `property_id`, while `reservation_date` acts as the clustering key.

To avoid double-bookings, Cassandra Lightweight Transactions (LWT) are used with the `IF NOT EXISTS` clause. These transactions rely on the Paxos consensus protocol to guarantee that only one reservation can be created for a given property and date.

3 Distributed System Features

Several distributed database concepts were implemented during the project:

- **Denormalization:** Data is modeled according to query patterns rather than relational normalization principles.
- **High Availability:** A three-node Cassandra cluster provides fault tolerance and data replication.
- **Concurrency Control:** Lightweight Transactions prevent race conditions during simultaneous booking attempts.
- **Scalability:** Cassandra's distributed architecture allows horizontal scaling by adding additional nodes.

4 Problems Encountered

Several challenges were encountered during the development of the project.

The main difficulty was preventing double-bookings when multiple users attempted to reserve the same property at the same time. This issue was solved using Cassandra Lightweight Transactions (LWT).

Another challenge was adapting the database schema to Cassandra's NoSQL model. Unlike relational databases, Cassandra requires a denormalized design optimized for specific queries.

The configuration of the three-node Cassandra cluster also required attention, as all nodes needed to communicate correctly before the application could start.

Finally, database operations occasionally caused the graphical interface to freeze. This problem was resolved by executing database requests in background threads.

5 Conclusion

RentCluster demonstrates how distributed database technologies can be used to build a reliable reservation system. By combining Apache Cassandra, Lightweight Transactions, and a distributed cluster architecture, the application successfully prevents double-bookings while maintaining scalability and high availability. The project provided practical experience with NoSQL data modeling, distributed consensus mechanisms, and concurrency management in real-world systems.